

Feature Engineering

Feature engineering is the process of transforming raw observational columns into a richer representation that exposes the statistical patterns a machine learning model needs to detect. Raw columns such as `serve_type` or `spin_intensity` each encode a single serve attribute in isolation, yet the tactical effectiveness of a serve is almost never attributable to any one attribute alone. A short backspin serve to the forehand corner carries a different strategic meaning from a short backspin serve to the body, even though both share the same spin type and length. This notebook constructs features that capture game state pressure, interactions among serve attributes, opponent characteristics, and the historical performance of specific serve combinations, thereby giving the model the context it needs to learn meaningful associations.

All features constructed here are restricted to information that is available before the serve is executed. This constraint is not just a methodological preference, it is a deployment requirement. The model trained in the next notebook must function as a real time decision support tool, which means it cannot rely on any variable whose value depends on the serve outcome or the subsequent rally. Every engineering decision in this notebook is evaluated against that constraint as to attempt to be as helpful in a real match scenario.

```
import pandas as pd
import numpy as np

df = pd.read_csv("../data/table_tennis_serves.csv")

df.head()
```

	serve_type	spin_type	spin_intensity	serve_length	placement_zone	toss_height	contact_point	match_id	game_id
0	backhand	float/no-spin	1	long	middle_FH	medium	body_center	1	1
1	backhand	float/no-spin	1	long	elbow	medium	body_center	1	1
2	backhand	float/no-spin	2	long	elbow	low	hip_level_backhand	1	1
3	backhand	float/no-spin	1	long	elbow	medium	hip_level_backhand	1	1
4	backhand	float/no-spin	1	short	elbow	low	body_center	1	1

5 rows × 25 columns



Data Validation

Before constructing any engineered features, we inspect the raw dataset to confirm that it loaded correctly and that the outcome column exhibits a plausible distribution. Verifying the row count, column set, and class balance at this stage is important because any loading failure or column mismatch would propagate

silently through all subsequent transformations, making the eventual error much harder to diagnose. We examine the value counts for `point_outcome` in particular because an unexpected class imbalance, for example a 90/10 split, would change the modeling strategy in the next notebook by requiring resampling or a modified loss function.

```
original_shape = df.shape
print("Dataset shape:", original_shape)
print()
print("Value counts for point_outcome:")
print(df["point_outcome"].value_counts())
```

Dataset shape: (500, 25)

Value counts for point_outcome:

point_outcome

lost 272

won 228

Name: count, dtype: int64

Target Variable Construction

The modeling objective is binary classification, predicting whether the server wins the point. This is encoded as `point_won`, where a value of 1 indicates that the server won the point and a value of 0 indicates that the server lost it. Binary integer encoding is preferred over retaining the raw string `point_outcome` column because it is directly interpretable by standard classification algorithms and evaluation metrics including ROC-AUC, log loss, and the Brier score. Using a binary numeric column also ensures that we can compute correlations, averages, and win rates as straightforward arithmetic operations without string comparisons at every step.

```
df["point_won"] = (df["point_outcome"] == "won").astype(int)
```

Data Leakage Identification

Data leakage occurs when a feature used during model training encodes information that would not be available at the time a prediction must be made in deployment. In this dataset, several columns describe events that occur after the serve has been executed, how the opponent returned the ball, how long the rally lasted, and how the point ultimately ended. Including any of these variables would produce artificially optimistic cross-validation scores during training because the model could exploit a direct association between the post-serve outcome description and the point result. In deployment, however, none of these values exist at the moment of the serve decision, so the model would receive missing or fabricated inputs and would fail in an unpredictable way. We identify and exclude these variables explicitly so that the feature set passed to the modeling notebook is provably free of leakage.

```
leakage_features = [
    "return_type",
    "return_quality",
    "return_placement",
    "rally_length",
    "point_end_type",
    "rally_type_achieved",
    "chop_rally_outcome"
]
leakage_features
```

```
['return_type',
 'return_quality',
 'return_placement',
 'rally_length',
 'point_end_type',
 'rally_type_achieved',
 'chop_rally_outcome']
```

Game State Features

The score at the time of a serve carries substantial tactical meaning that extends beyond the raw server and receiver score values. A server who is tied at 10-10 faces different psychological and strategic pressures than one who leads 8-4, even though both situations involve ten server points. We construct a set of binary flags and a continuous margin variable to capture these game-state conditions in a form that a model can use directly. The `is_tied` flag identifies moments of neutral high-stakes parity. The `is_trailing` and `is_leading` flags encode the direction of the score differential, enabling the model to detect whether servers adopt more aggressive or more conservative tactics when they are behind or ahead. The `is_late_game` flag marks the final stretch of a standard game, where fatigue and heightened attention may alter serve selection. The `is_deuce_or_later`, `is_game_point_for_server`, and `is_game_point_against_server` flags capture the highest pressure moments, where a single point determines who wins the game and where serve choice is most consequential.

```
df["score_margin"] = df["server_score"] - df["receiver_score"]
df["total_points_played_in_game"] = df["server_score"] + df["receiver_score"]
df["is_tied"] = (df["server_score"] == df["receiver_score"]).astype(int)
df["is_trailing"] = (df["server_score"] < df["receiver_score"]).astype(int)
df["is_leading"] = (df["server_score"] > df["receiver_score"]).astype(int)
df["is_late_game"] = (df["total_points_played_in_game"] >= 16).astype(int)
df["is_deuce_or_later"] = ((df["server_score"] >= 10) & (df["receiver_score"] >= 10)).astype(int)
df["is_game_point_for_server"] = ((df["server_score"] >= 10) & (df["server_score"] > df["receiver_score"])).astype(int)
df["is_game_point_against_server"] = ((df["receiver_score"] >= 10) & (df["receiver_score"] > df["server_score"])).astype(int)
```

Interaction Features

Game-state flags and serve attributes are each informative individually, but certain combinations carry predictive signal that neither variable encodes alone. We construct three interaction features to capture these joint effects. The `spin_x_looper` feature is the product of continuous spin intensity and the binary indicator for a looping opponent style. The hypothesis is that a high-spin serve is disproportionately effective against looping opponents because loopers are trained to exploit incoming spin, and an unexpectedly heavy or deceptive spin challenges their timing in ways that are specific to their style. The `score_margin_abs` feature measures how far the score is from a tie in a direction-agnostic way, useful for capturing pressure that arises both when the server is well ahead and when the server is well behind. The `is_high_pressure` flag collapses three correlated binary flags into a single indicator for any situation in which a game-deciding point could occur, reducing multicollinearity without sacrificing the information those flags collectively encode.

```
# Opponent style dummies are created here because spin_x_looper depends on opponent_is_looper
df["opponent_is_looper"] = (df["opponent_style"] == "looper").astype(int)
df["opponent_is_chopper"] = (df["opponent_style"] == "chopper").astype(int)
df["opponent_is_attacker"] = (df["opponent_style"] == "attacker").astype(int)

# Spin intensity amplified against looping opponents
df["spin_x_looper"] = df["spin_intensity"] * df["opponent_is_looper"]

# Pressure distance: how far from a tie, regardless of direction
df["score_margin_abs"] = df["score_margin"].abs()

# Unified high-pressure flag combining three correlated game-state conditions
df["is_high_pressure"] = (
    (df["is_deuce_or_later"] == 1) |
    (df["is_game_point_for_server"] == 1) |
    (df["is_game_point_against_server"] == 1)
).astype(int)

print("Interaction features added.")
print(df[["spin_x_looper", "score_margin_abs", "is_high_pressure"]].describe())
```

Interaction features added.

	spin_x_looper	score_margin_abs	is_high_pressure
count	500.000000	500.000000	500.000000
mean	0.808000	2.644000	0.120000
std	1.072117	2.453835	0.325287
min	0.000000	0.000000	0.000000
25%	0.000000	1.000000	0.000000
50%	0.000000	2.000000	0.000000
75%	2.000000	4.000000	0.000000
max	3.000000	10.000000	1.000000

Opponent Encoding

The `opponent_skill_level` column contains categories that have a natural and meaningful order: a beginner opponent is substantially less challenging than an intermediate opponent, and an intermediate opponent is less challenging than an advanced or expert opponent. Ordinal encoding maps these levels to integers 1 through 4, preserving the rank ordering so that models can learn a monotonic relationship between opponent difficulty and serve effectiveness without treating each level as a completely independent category. A nominal encoding such as one-hot encoding would discard this ordering information entirely, forcing the model to infer the rank structure from data alone rather than incorporating it directly. We use a dictionary mapping followed by a `fillna` to handle any unexpected values gracefully, defaulting to the intermediate level.

```
skill_map = {"beginner": 1, "intermediate": 2, "advanced": 3, "expert": 4}
df["opponent_skill_numeric"] = df["opponent_skill_level"].map(skill_map).fillna(2)

print("Value counts for opponent_skill_numeric:")
print(df["opponent_skill_numeric"].value_counts().sort_index())
```

Value counts for opponent_skill_numeric:

opponent_skill_numeric

1.0 68

2.0 310

3.0 122

Name: count, dtype: int64

Serve Combination Features

A serve is not a single categorical choice but a simultaneous selection across multiple attributes: serve type, spin type, length, and placement zone. The tactical effect of these attributes is multiplicative rather than additive. A short backspin serve to the forehand corner produces a qualitatively different challenge for the receiver than a short backspin serve to the body, even though those two serves differ only in placement. Modeling each attribute as an independent predictor cannot capture these joint effects. We construct explicit string concatenations of attribute pairs and of all four attributes together to create serve combination identifiers that the model can treat as categorical variables. This allows the model to learn win-rate patterns at the combination level rather than being forced to decompose those patterns into independent attribute-level contributions that may not exist.

```
df["serve_spin_combo"] = df["serve_type"] + "_" + df["spin_type"]
df["serve_length_spin_combo"] = df["serve_length"] + "_" + df["spin_type"]
df["serve_placement_combo"] = df["serve_type"] + "_" + df["placement_zone"]
df["full_serve_combo"] = (
    df["serve_type"] + "_" +
    df["spin_type"] + "_" +
    df["serve_length"] + "_" +
```

```
df["placement_zone"]
)
```

Spin Intensity Flags

Spin intensity is measured on a continuous numeric scale, but its tactical implications often manifest at discrete thresholds rather than as a smooth gradient. A heavy spin serve requires the receiver to make fundamentally different contact angle adjustments compared to a no spin serve, and the qualitative difference between a very heavy spin and a moderate spin is more practically significant than the numerical difference between the same integers would suggest. Discretizing spin intensity into binary flags for heavy spin (intensity at or above 3) and low spin (intensity at or below 1) allows the model to detect these threshold effects directly rather than having to learn them from a continuous variable. The `spin_length_interaction` string feature combines the discrete spin level with serve length to identify specific tactical combinations such as a heavy spin short serve, which is recognized in competitive table tennis as a particularly difficult return because it constrains the receiver's contact options while maximizing the influence of spin.

```
df["is_heavy_spin"] = (df["spin_intensity"] >= 3).astype(int)
df["is_low_spin"] = (df["spin_intensity"] <= 1).astype(int)
df["spin_length_interaction"] = df["spin_intensity"].astype(str) + "_" + df["serve_length"]
```

Historical Combo Statistics

The historical win rate of a specific serve combination is valuable predictive information, but a raw win rate computed from a small number of observations is statistically unreliable. A serve combination attempted only twice could show a 100% win rate purely by chance, yet that estimate carries no more confidence than a coin flip. We address this problem with two complementary features that jointly represent both the observed performance and its reliability. The `combo_win_rate` is the observed fraction of won points across all historical uses of a given full serve combination. The `combo_reliability` score is the fraction of a 30 attempt reference sample that has been observed, capped at 1.0, such that a combination with 15 attempts receives a reliability of 0.5 and one with 30 or more receives 1.0. This reliability score enables both the model and the recommendation system to discount win rates based on few observations while giving full weight to those based on 30 or more, which represents a pragmatic sample size threshold for stable proportion estimates.

```
combo_summary = (
    df.groupby("full_serve_combo")
    .agg(
        combo_attempts=("point_won", "count"),
        combo_win_rate=("point_won", "mean")
    )
    .reset_index()
)
```

```
df = df.merge(combo_summary, on="full_serve_combo", how="left")
df["combo_reliability"] = np.minimum(df["combo_attempts"] / 30, 1)
```

Feature Validation

Three programmatic sanity checks are applied before saving the engineered dataset. The first check compares the column count against the original dataset to confirm that all features were appended as expected, a discrepancy would indicate a naming collision, a failed merge, or a silent overwrite. The second check scans for null values in the engineered columns, because any null present in a feature used by the model will cause a runtime error during inference and because null values in a training dataset can distort learned coefficients if handled inconsistently. The third check computes the absolute Pearson correlation of each new numeric feature with the target variable and displays the top 10, which serves as a preliminary diagnostic for feature relevance and can also help identify any feature with suspiciously high correlation that might indicate residual data leakage that escaped the explicit exclusion step.

```
# Check 1: Column count comparison
engineered_count = df.shape[1] - original_shape[1]
print(f"Original column count : {original_shape[1]}")
print(f"Current column count  : {df.shape[1]}")
print(f"Engineered features   : {engineered_count}")
print()

# Check 2: Null check on new columns only
new_columns = df.columns[original_shape[1]:].tolist()
null_counts = df[new_columns].isnull().sum()
print("Null values in engineered columns:")
print(null_counts[null_counts > 0] if null_counts.any() else " None. All engineered columns are ")
print()

# Check 3: Top-10 numeric features by absolute correlation with point_won
# Exclude point_won itself from the feature list to avoid a duplicate column when building the combo
numeric_new = [
    c for c in df[new_columns].select_dtypes(include=[np.number]).columns.tolist()
    if c != "point_won"
]
correlations = (
    df[numeric_new + ["point_won"]]
    .corr()["point_won"]
    .drop("point_won", errors="ignore")
    .abs()
    .sort_values(ascending=False)
    .head(10)
)
print("Top-10 engineered features by |correlation| with point_won:")
print(correlations.to_string())
```

Original column count : 25
Current column count : 52
Engineered features : 27

Null values in engineered columns:
None -- all engineered columns are complete.

Top-10 engineered features by |correlation| with point_won:

combo_win_rate	0.544892
score_margin	0.231540
opponent_skill_numeric	0.196006
is_trailing	0.192401
is_leading	0.157273
opponent_is_looper	0.129113
is_heavy_spin	0.123706
is_low_spin	0.099597
is_high_pressure	0.069694
is_late_game	0.069603

Feature Summary

The table below consolidates all engineered features by category, together with their data types and Pearson correlations with the target variable. This summary provides a structured reference for the feature set that will be passed to the modeling notebook and serves as documentation for any future analyst who needs to understand what was constructed and why. Features listed with dtype `object` are categorical string variables that will be handled by the one-hot encoding step in the preprocessing pipeline of the modeling notebook; they do not have a meaningful numeric correlation with the target and are marked accordingly.

```
feature_groups = {  
    "Game State": [  
        "score_margin", "total_points_played_in_game", "is_tied", "is_trailing",  
        "is_leading", "is_late_game", "is_deuce_or_later",  
        "is_game_point_for_server", "is_game_point_against_server"  
    ],  
    "Interaction": ["spin_x_looper", "score_margin_abs", "is_high_pressure"],  
    "Serve Combinations": [  
        "serve_spin_combo", "serve_length_spin_combo",  
        "serve_placement_combo", "full_serve_combo"  
    ],  
    "Spin Flags": ["is_heavy_spin", "is_low_spin", "spin_length_interaction"],  
    "Opponent": [  
        "opponent_is_looper", "opponent_is_chopper",  
        "opponent_is_attacker", "opponent_skill_numeric"  
    ],  
    "Combo Statistics": ["combo_attempts", "combo_win_rate", "combo_reliability"]  
}
```



```

rows = []
for group, features in feature_groups.items():
    for feature in features:
        if feature not in df.columns:
            continue
        dtype = str(df[feature].dtype)
        if pd.api.types.is_numeric_dtype(df[feature]):
            win_rate_corr = round(df[feature].corr(df["point_won"]), 4)
        else:
            win_rate_corr = "N/A"
        rows.append({"group": group, "feature": feature, "dtype": dtype, "win_rate_corr": win_rate_corr})

summary_df = pd.DataFrame(rows, columns=["group", "feature", "dtype", "win_rate_corr"])
print(summary_df.groupby("group").apply(lambda x: x[["feature", "dtype", "win_rate_corr"]].to_string(

```

```

group
Combo Statistics          feature  dtype win_rate_corr\n  co...
Game State                feature dtype win_rate_co...
Interaction               feature dtype win_rate_corr\n  spin...
Opponent                  feature  dtype win_rate_corr\n...
Serve Combinations       feature  dtype win_rate_corr\n...
Spin Flags                feature  dtype win_rate_corr\n...

```

```

c:\Users\shrey\AppData\Local\Programs\Python\Python313\Lib\site-
packages\numpy\lib\_function_base_impl.py:3045: RuntimeWarning: invalid value encountered in
divide

```

```

    c /= stddev[:, None]

```

```

c:\Users\shrey\AppData\Local\Programs\Python\Python313\Lib\site-
packages\numpy\lib\_function_base_impl.py:3046: RuntimeWarning: invalid value encountered in
divide

```

```

    c /= stddev[None, :]

```

```

C:\Users\shrey\AppData\Local\Temp\ipykernel_19860\1127378735.py:33: DeprecationWarning:
DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a

```

```

`include_groups=False` to exclude the groupings or explicitly select the grouping columns after
groupby to silence this warning.

```

```

    print(summary_df.groupby("group").apply(lambda x: x[["feature", "dtype",
"win_rate_corr"]].to_string(index=False)).to_string())

```

Save and Verify

The fully engineered dataset is written to the [data/processed/](#) directory so that the modeling notebook can load it directly without repeating any of the transformation steps. Writing to a separate processed directory rather than overwriting the raw file preserves the original data for auditing and for rerunning this notebook with different engineering choices. We then reload the saved file and confirm that the row and column counts match the in memory dataframe exactly, which guards against silent truncation or encoding errors that can occur during the CSV write operation on large files with special characters.

```
output_path = "../data/processed/table_tennis_serves_features.csv"
df.to_csv(output_path, index=False)
```

```
import os

saved_df = pd.read_csv(output_path)
print("File saved successfully.")
print(f" Path : {os.path.abspath(output_path)}")
print(f" Shape : {saved_df.shape[0]} rows x {saved_df.shape[1]} columns")
```

File saved successfully.

Path : c:\Users\shrey\OneDrive\Desktop\Personal Projects\tt\Table-Tennis-Serve-Analysis\data\processed\table_tennis_serves_features.csv
Shape : 500 rows x 52 columns

The processed dataset is now ready for consumption by the modeling notebook. All features included here are pre-serve variables, confirming that no outcome information has been allowed to influence the predictor space. The engineering decisions documented above collectively provide the model with game context, tactical combination identifiers, and historical performance statistics that raw columns alone could not supply.

Feature Engineering Quality Assurance

As a final programmatic check, we assert that all required engineered columns are present in the saved dataframe and that none of them contain null values. These assertions function as a formal contract between this notebook and the modeling notebook: if any assertion fails, execution halts immediately with a descriptive error message rather than propagating a silent defect into the model training pipeline. This approach makes the dependency between the two notebooks explicit and testable, which is important for reproducibility when the pipeline is rerun on new data.

```
required_engineered = [
    'score_margin', 'total_points_played_in_game', 'is_tied', 'is_trailing', 'is_leading',
    'serve_spin_combo', 'serve_length_spin_combo', 'serve_placement_combo', 'full_serve_combo',
    'combo_attempts', 'combo_win_rate', 'combo_reliability', 'point_won'
]
missing = [c for c in required_engineered if c not in df.columns]
assert not missing, f'Missing engineered columns: {missing}'
assert df[required_engineered].isnull().sum().sum() == 0, 'Nulls found in required engineered columns'
print('Quality assurance checks passed: all required engineered features exist and contain no nulls')
```

Quality assurance checks passed: all required engineered features exist and contain no nulls.

Conclusion for Notebook 2

This notebook converts the raw serve dataset into a model ready feature set that better reflects the tactical context of each point. By engineering score state indicators, pressure flags, opponent encodings, serve combination variables, spin thresholds, and historical combination statistics, the dataset now captures information that is more meaningful than the original raw columns alone. Just as importantly, each feature is restricted to information available before the serve occurs, which helps prevent data leakage and preserves the realism of the prediction task. The final validation and quality assurance checks confirm that the engineered columns were created successfully, contain no missing values, and were saved correctly for the next stage of the pipeline. With this processed dataset in place, the modeling notebook can focus on training, evaluating, and interpreting predictive models for serve effectiveness.